Predicting stock prices with LSTM Networks

David Duemig

Abstract

In this project we explore a method of predicting stock price movements. We apply a long short term memory (LSTM) based recurrent neural network (RNN) to predict whether the S&P 500 will increase (up) or decrease (down) over the next trading month, using the features volatility, return and trading volume as three sets of time sequence data. We use accuracy and the area under the ROC curve (AUC) to tune hyper-parameters and to measure the performance of our prediction. It turns out that the LSTM models perform much like the corresponding baseline linear classification models, and that neither beats the naive rule of always predicting "up". Test AUCs lie between 0.44 and 0.62, and a timing strategy built on the forecasts does not beat buying and holding after costs.

1 Introduction

Artificial Intelligence is changing virtually every aspect of our lives. Today's algorithms accomplish tasks that until recently only expert humans could perform. As it relates to finance, this is an exciting time to adopt a disruptive technology that will transform how investment decisions are made on a broad scale.

Models that explain the returns of individual stocks generally use company and stock characteristics, e.g., the market prices of financial instruments and companies' accounting data. These characteristics can also be used to predict expected stock returns out-of-sample. Most studies use simple linear models to form these predictions [1, 2]. An increasing body of academic literature documents that more sophisticated tools from the Machine Learning (ML) and Deep Learning (DL) repertoire, which allow for nonlinear predictor interactions, can improve the stock return forecasts [3, 4, 5].

The main goal of this project is to investigate whether modern DL techniques can be utilized to more efficiently predict the movements of the stock market. Specifically, we train LSTM networks with time series price-volume data and compare their out-of-sample return predictability with the performance of simple logistic regressions, our baseline models.

2 Related work

The financial industry has been slow to integrate DL techniques into their systems. Moreover, successful financial models are usually proprietary, and it is therefore difficult to find compelling literature with state-of-the-art performance on our specific task. However, the academic community has started to acknowledge the advances in Financial Machine Learning. Consequently, there are increasingly more papers that attempt to apply DL methods to tackle the immensely complex problem of stock market prediction. One common theme amongst these works is the choice of an LSTM based recurrent neural network. For example, Gao applied an LSTM network to forecast the stock markets in six different industries in the US, using 359 stock features as input [6]. Nelson et al generated 175 features for each 15 minutes of stock data using an LSTM network to predict whether the stock price will go up or down [8]. A third study by Chen et al uses historical stock price data to predict whether a stock price will rise, fall or remain in a day, using an LSTM architecture as well [7].

Inspired by the above papers, this project also proposes a model based on an LSTM architecture to predict price movements using dominant predictive signals that include variations on momentum, volatility and liquidity as input. These input features have been shown to have widespread predictive power.

3 Dataset and Features

3.1 Raw Data

The dataset provides daily quotes for the S&P 500 going back to 1980 and is downloaded with `yfinance` (the older Yahoo Finance reader is discontinued), covering January 1980 to December 2025. It comes with the open, high, low, close and volume for each day. In total the raw dataset consists of approximately 11,595 rows.

3.2 Features

The feature set includes rolling (i) logarithmic returns, (ii) volatilities and (iii) trading volumes for the past i trading week(s). All three are stationary, which matters because the index level and the traded volume both grow by large factors over four decades.

First, daily logarithmic returns can be computed as shown below:

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right). \tag{1}$$

Since logarithmic returns are additive over time, we can simply sum over the corresponding number of daily returns to compute the rolling logarithmic returns for each of the past i trading weeks. This gives us 23 features.

Next, rolling volatilities for each of the past i trading weeks can be calculated as the standard deviation of the daily returns over the window:

$$\sigma_{i,t} = \mathrm{sd}\left(r_{t-5i+1}, \ldots, r_t\right). \tag{2}$$

This provides us with additional 23 features.

Lastly, instead of the raw rolling average volume, which trends strongly over the sample, we use log volume relative to its trailing one-year mean,

$$\mathrm{volume}_{i,t} = \overline{\ln V}_{t,\,5i} - \overline{\ln V}_{t,\,252}, \tag{3}$$

which is stationary and adds another 23 features. This results in a total of 69 features. The baselines use these month-end features; the LSTM instead reads the most recent 60 trading days of the corresponding daily series.

3.3 Scaling

In the next step, all features are rescaled before training in order to (i) make the contours of the cost function more symmetrical in shape and thus easier to optimize, (ii) make training less sensitive to the scale of features and (iii) ensure regularization does not behave differently for different scaling. Normalization and Standardization are two well-known methods for rescaling data. We try both and keep the one that works best for our application, which is standardization. The scaler is fitted inside each cross-validation fold on the training part only, so no scaling information leaks forward.

3.3.1 Standardization

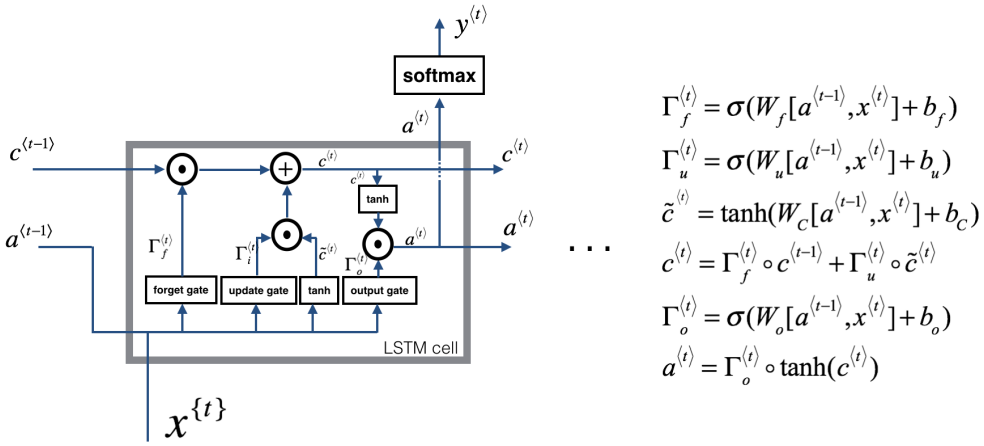
Standardization rescales the data to have a mean (μ) of 0 and a standard deviation (σ) of 1 (unit variance):

$$X_z^{(i)} = \frac{X^{(i)} - \bar{X}}{s_X}. \tag{4}$$

3.3.2 Normalization

On the other hand, Normalization scales all numeric variables in the range $[0, 1]$:

$$X_{norm}^{(i)} = \frac{X^{(i)} - X_{min}}{X_{max} - X_{min}}. \tag{5}$$



$$\Gamma_f^{\langle t\rangle} = \sigma(W_f[a^{\langle t-1\rangle}, x^{\langle t\rangle}] + b_f)$$
$$\Gamma_u^{\langle t\rangle} = \sigma(W_u[a^{\langle t-1\rangle}, x^{\langle t\rangle}] + b_u)$$
$$\tilde{c}^{\langle t\rangle} = \tanh(W_C[a^{\langle t-1\rangle}, x^{\langle t\rangle}] + b_C)$$
$$c^{\langle t\rangle} = \Gamma_f^{\langle t\rangle} \circ c^{\langle t-1\rangle} + \Gamma_u^{\langle t\rangle} \circ \tilde{c}^{\langle t\rangle}$$
$$\Gamma_o^{\langle t\rangle} = \sigma(W_o[a^{\langle t-1\rangle}, x^{\langle t\rangle}] + b_o)$$
$$a^{\langle t\rangle} = \Gamma_o^{\langle t\rangle} \circ \tanh(c^{\langle t\rangle})$$

Figure 1: The LSTM cell and its gate equations. A sequence of daily feature vectors feeds one LSTM layer whose output passes through a single output unit.

3.4 Separating Test Data

Since K-Fold Cross Validation fails in finance [9], we instead use walk-forward cross-validation for our model development (hyper-parameter tuning). Furthermore, to provide an unbiased evaluation of our final model, we withhold the last 20% of our data as a separate test set. This set is only used once our model is completely trained. Because each label looks one month ahead, a training month whose label window reaches a validation month is purged, and a further embargo of 1 month is dropped before each validation block, so no future information leaks into the past.

3.5 Label Distribution

The labels indicate whether the market had a positive or negative return on a given month, using one non-overlapping observation per month so that neighbouring labels do not share returns. The classes are unbalanced in favour of "up": 62.0% of the development months and 68.9% of the test months are up. This imbalance is central to reading the results, because a model that always predicts "up" already scores 68.9% on the test set.

4 Models

4.1 Baseline Models

To provide linear baselines for reference, simple logistic regressions are implemented. As described in Section 3.4, we use walk-forward cross-validation for model development and then evaluate the performance on the hold-out test set. As part of the optimization, the penalties Ridge, Lasso and Elastic Net, and their corresponding hyperparameters, are tuned. It turns out that the standardized data is best suited for this task. We also add a gradient-boosting classifier as a second, nonlinear baseline, tuned on the same folds, and we report the naive always-up rule as the benchmark that every model has to beat.

4.2 LSTM Networks

Given the recent success of RNNs in the area of sequential data, and taking into account the temporal sequence of our data, we apply such an architecture to our prediction task. Specifically, we use an LSTM network, which is a special type of recurrent neural network that works much better than the standard version for many tasks. One of the appeals of LSTMs is the idea that they are able to learn long-term dependencies. Our network structure is depicted in the following figure.

The key to LSTMs is the cell state, the horizontal line that runs through the top of the diagram (Figure 1). With only a few linear interactions, it is easy for information to flow through. The LSTM can remove or add information to the cell state, carefully controlled by three gates. The first gate

Table 1: Summary of model results. The baseline is the elastic-net logistic regression; accuracy cells show development / test accuracy. The test set has 106 months with an up rate of 68.9%.

		Dev / Test accuracy		Test AUC	
Model	Features	LSTM	Baseline	LSTM	Baseline
1	Volatility	63.5% / 67.9%	58.9% / 68.9%	0.49	0.48
2	Return	63.5% / 68.9%	59.4% / 66.0%	0.54	0.52
3	Volume	63.5% / 68.9%	58.0% / 55.7%	0.45	0.51
4	Volatility + Return	63.5% / 68.9%	58.9% / 67.9%	0.53	0.55
5	Volatility + Volume	63.8% / 68.9%	56.3% / 68.9%	0.44	0.51
6	Return + Volume	63.4% / 68.9%	56.1% / 64.1%	0.48	0.51
7	Volatility + Return + Volume	63.5% / 68.9%	53.2% / 68.9%	0.53	0.62

decides which information is discarded from the cell state; this decision is made by a sigmoid layer called the forget gate. The second gate decides which new information will be stored in the cell state, through a sigmoid update gate and a tanh layer that creates a vector of candidate values. Finally, the output gate decides which part of the cell state is read out [10].

4.3 Experiment Design

As aforementioned, we use walk-forward cross-validation for model development and then evaluate on the hold-out test set. Due to the nature of our features, we apply an LSTM that reads a 60-day sequence for each month and outputs the probability of an up month through a single sigmoid unit. The network is trained with early stopping. Because the sample is small, the architecture is kept small and is not tuned; we instead average over 10 random seeds and report the mean and the spread.

4.4 Experiment Results

We have three sets of time sequence data: (1) volatility, (2) return and (3) trading volume. By combining these feature sets, 7 models are created and compared with their corresponding baseline model. As mentioned above, we use accuracy and AUC as our evaluation metrics, now with 95% block-bootstrap intervals over months. Table 1 summarizes the results.

From a trading perspective we are interested in maximizing accuracy, as this tells us how well the model does on both up and down months. However, as described in Section 3.5, there is a significant imbalance in the data, so we must take accuracy with a grain of salt, and the AUC score is a better guide because it is insensitive to class imbalance. All models have a test AUC close to 0.5, from 0.44 to 0.62, which is an unsatisfactory result, and the best test accuracy is 68.9%, equal to the always-up rate; no model beats always-up by a margin whose interval excludes zero. One of the twenty-one model and feature-set combinations, the elastic net on all three families, has an AUC interval that just clears 0.5, which is close to what chance alone produces across that many combinations. The gradient-boosting baseline behaves the same way, with test AUCs between 0.47 and 0.58. One possible reason is that our features do not contain enough information to predict future returns. Moreover, the input data is very noisy, and it is difficult for the classifier to generalize to other periods.

We also test a simple timing strategy: hold the index next month when the predicted probability of an up month exceeds a threshold chosen on validation data only, otherwise hold cash, after a transaction cost of 10 basis points. Table 2 compares it with buying and holding. Buy-and-hold earns 12.8% a year at a Sharpe ratio of 0.85. The best timing strategy reaches a Sharpe of 1.00, but it is the best of twenty-one combinations and most strategies simply stay invested and recover buy-and-hold, so the edge is not evidence of real timing skill.

5 Conclusion and Future Work

From this project we can conclude that stock price forecasting remains a difficult task, even after simplifying the problem to a binary price trend, cleaning up the validation, and applying a sequence model that reads a real history. The LSTM, the linear baseline and the gradient-boosting model all

Table 2: Annualised test performance, buy-and-hold against the best timing strategy per model family, after costs.

Strategy	Ann. return	Ann. vol	Sharpe	Max drawdown
Buy and hold	12.8%	15.7%	0.85	-24.8%
Elastic net (set 1)	12.8%	15.7%	0.85	-24.8%
Gradient boosting (set 3)	14.3%	14.7%	0.99	-17.7%
LSTM (set 3)	14.7%	14.9%	1.00	-22.5%

sit close to the always-up rule, and the timing strategy does not beat buying and holding after costs. Nonetheless, this information is useful in guiding future work. We have shown that it might need more than price, volume and volatility data to predict future returns. Potential future directions include enriching the dataset with non-technical features such as fundamentals; changing the objective to a continuous return or a quantile rather than a sign; exploring other models such as attention; and studying a longer or international sample so that the test set is not dominated by a single rising market.

Acknowledgements

This project began as a course project for CS230 Deep Learning. I thank the teaching team and my advisor Sagar Honnungar.

References

[1] Blitz, D. & Vidojevic, M. (2018), 'The Characteristics of Factor Investing', SSRN Electronic Journal, https://ssrn.com/abstract=3206798.

[2] Lewellen, J. (2015), 'The Cross-section of Expected Stock Returns', Critical Finance Review, vol. 4, no. 1, pp. 1–44, https://ssrn.com/abstract=2511246.

[3] Freyberger, J., Neuhierl, A. & Weber, M. (2017), 'Dissecting characteristics nonparametrically', CESifo Working Paper 6391, https://ssrn.com/abstract=2935432.

[4] Gu, S., Kelly, B.T. & Xiu, D. (2018), 'Empirical Asset Pricing via Machine Learning', Chicago Booth Research Paper 18-04, https://ssrn.com/abstract=3159577.

[5] Rapach, D., Strauss, J., Tu, J. & Zhou, G. (2018), 'Industry Return Predictability: A Machine Learning Approach', SSRN Electronic Journal, https://ssrn.com/abstract=3120110.

[6] Gao, Q. (2016), 'Stock market forecasting using recurrent neural network', University of Missouri, Columbia.

[7] Chen, K., Zhou, Y. & Dai, F. (2015), 'A LSTM-based method for stock returns prediction', Proceedings of the 2015 IEEE International Conference on Big Data, pp. 2823–2824.

[8] Nelson, D.M., Pereira, A.C. & de Oliveira, R.A. (2017), 'Stock markets movement prediction with LSTM neural networks', Proceedings of the 2017 International Joint Conference on Neural Networks, pp. 1419–1426.

[9] Lopez de Prado, M. (2018), 'Advances in Financial Machine Learning', Wiley, New Jersey.

[10] Olah, C. (2015), 'Understanding LSTM Networks', http://colah.github.io/posts/2015-08-Understanding-LSTMs/.

Github Repository

https://github.com/duemig/Stanford-Project-Predicting-stock-prices-using-a-LSTM-Network